Securing Sensitive Communications: A Look at End-to-End Encrypted Chat Application

COMPUTER NETWORKS LABORATORY
IT5511

AMRITHA S(2020506011)
MADHUMITHA R(2020506049)
NEHA M(2020506058)

Abstract - One of the most important features of a chat application is the ability to protect the privacy of its users. As more and more people rely on these apps to communicate and share sensitive information, it's crucial that the chat application implements robust security measures to keep user data safe. End-to-end encryption is a key aspect of this, as it ensures that only the intended recipients can read the messages being sent. Additionally, the chat application should have a user-friendly interface and be easy to use for all types of users. Overall, the goal of this report is to suggest a chat application that offers the highest level of security, so users can confidently use the app to discuss sensitive information.

I. Introduction:

Online chat has revolutionised the way we communicate, making it easier and faster for people to connect with one another from anywhere in the world. From instant messaging apps on our smartphones to video conferencing tools, online chat has become an integral part of our everyday lives. It has changed the way we work, learn, and socialise, making it possible for people to collaborate and share ideas in real-time. The convenience and flexibility of online chat has made it a go-to choice for both personal and professional communication.

In terms of technology, online chat has played a significant role in the development of instant messaging and other forms of real-time communication. The rise of instant messaging apps, such as WhatsApp and Telegram, has allowed people to communicate faster and more efficiently than ever before. Additionally, the growth of live streaming platforms and video conferencing tools has made it possible for people to have face-to-face conversations online, even when they are miles apart. The advancements in technology have also enabled online chat to be more accessible, allowing people to communicate through a variety of devices and platforms.

Despite the many benefits of online chat, it is not without its risks. The most significant concern is the security of the data that is being shared. With more and more personal and sensitive information being shared through chat apps, there is a growing risk of data breaches and cyber attacks. Hackers and cybercriminals are constantly finding new ways to access and exploit sensitive information, putting user's personal and financial data at risk. This is particularly concerning for businesses, which often rely on chat apps to share and store sensitive information.

The report, "Securing Sensitive Communications: A Look at End-to-End Encrypted Chat Applications" addresses this problem by focusing on chat applications that offer end-to-end encryption to protect user privacy and secure sensitive information being shared. End-to-end encryption is a security measure that ensures that only the intended recipients can read the messages being sent, making it much more difficult for hackers

and cybercriminals to access user data. By selecting a chat app that offers end-to-end encryption, individuals and businesses can feel confident that their data is being protected.

In conclusion, online chat has played an instrumental role in the way we communicate today, making it easier and more convenient than ever before. However, it also comes with security concerns. This project, "Securing Sensitive Communications: A Look at End-to-End Encrypted Chat Applications" is an essential exploration of the various chat apps that are currently available in the market. It aims to help individuals and businesses choose a secure and private way of communicating, which is of utmost importance in today's digital age. It suggests that the chat applications that offer end-to-end encryption is the best way to ensure the security of the data and personal information shared online

II. Background Literature:

i) Express.JS:

xpress.js is a popular JavaScript framework for building web applications. It is built on top of Node.js, which is a JavaScript runtime that allows developers to run JavaScript code on the server-side. Express.js is designed to make it easier to build web applications by providing a simple and flexible way to handle routing and middleware.

One of the key features of Express.js is its routing system. This allows developers to handle different HTTP requests and responses, such as GET, POST, and DELETE, by mapping them to specific routes in the application. This makes it easy to organize and structure the different parts of a web application, making it more manageable and easier to maintain.

Another important feature of Express.js is its middleware system. Middleware functions are small code blocks that are used to perform specific tasks, such as validating user input or handling authentication. Express.js has a large ecosystem of middleware functions available, which can be easily integrated into web applications to add additional

functionality. This makes it easy to build complex web applications with minimal code.

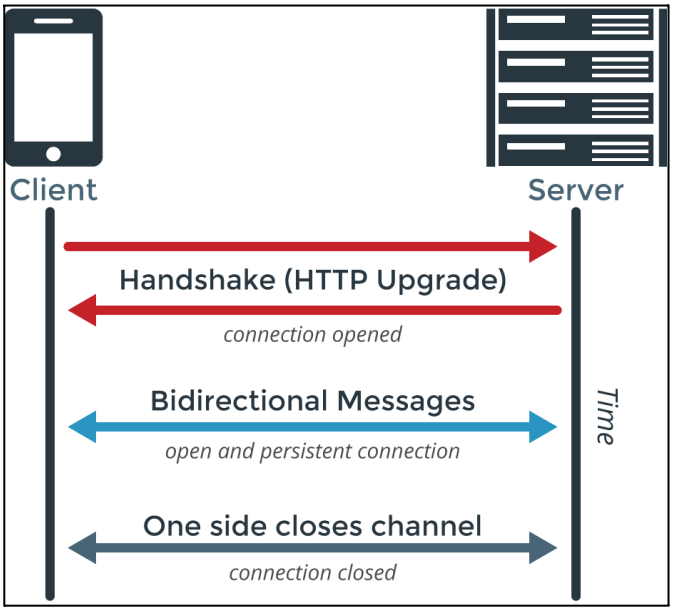
Express.js also provides support for templating engines, which are used to generate HTML pages dynamically. This is useful for building web applications that require dynamic pages, such as e-commerce or social media websites. Express.js works well with popular template engines like EJS, Handlebars and Pug, which can be easily integrated into an application.

In addition to its features, Express.js has a strong developer community which constantly provides support, documentation and tutorials. It makes it a reliable and widely used framework for building web applications, with many popular websites and apps like Uber, Accenture, and IBM using it to power their applications

ii) Web Socket:

WebSocket is a computer networking protocol that enables full-duplex communication over a single Transmission Control Protocol (TCP) connection. It is designed to be implemented in web browsers and web servers, but it can also be used by any client or server application. The WebSocket protocol allows for real-time, bidirectional communication between a client and a server, which means that both parties can send and receive data simultaneously. This is in contrast to traditional HTTP communication, which is based on a request-response model where the client must send a request and wait for a response from the server.

WebSockets are widely used in modern web applications to provide real-time functionality such as chat, notifications, and live updates. This makes them an essential tool for creating interactive and engaging user experiences. They are also used in gaming, stock trading platforms and many other fields where low-latency communication is crucial. The protocol's capability to establish a low-latency, bidirectional connection between a browser and a server enables a variety of real-time use cases to be built.



WebSockets use a handshake mechanism to establish a connection between a client and a server, which makes them secure and easy to implement. Once the connection is established, the WebSocket protocol uses an efficient binary format for data transfer, which reduces the overhead of encoding and parsing data, thus making it more efficient than HTTP. The WebSockets API is also very simple, allowing developers to create WebSockets applications using JavaScript

iii) TCP:

TCP (Transmission Control Protocol) is a transport-layer protocol that is used to establish a reliable connection between two devices in a computer network. It is used in conjunction with IP (Internet Protocol) to create the TCP/IP protocol suite, which is the foundation of all internet communication.

TCP is a connection-oriented protocol, which means that it establishes a reliable, point-to-point connection between two devices before any data is exchanged. When two devices want to communicate using TCP, they first go through a process called the "TCP three-way handshake." This process establishes a "virtual connection" between the devices and

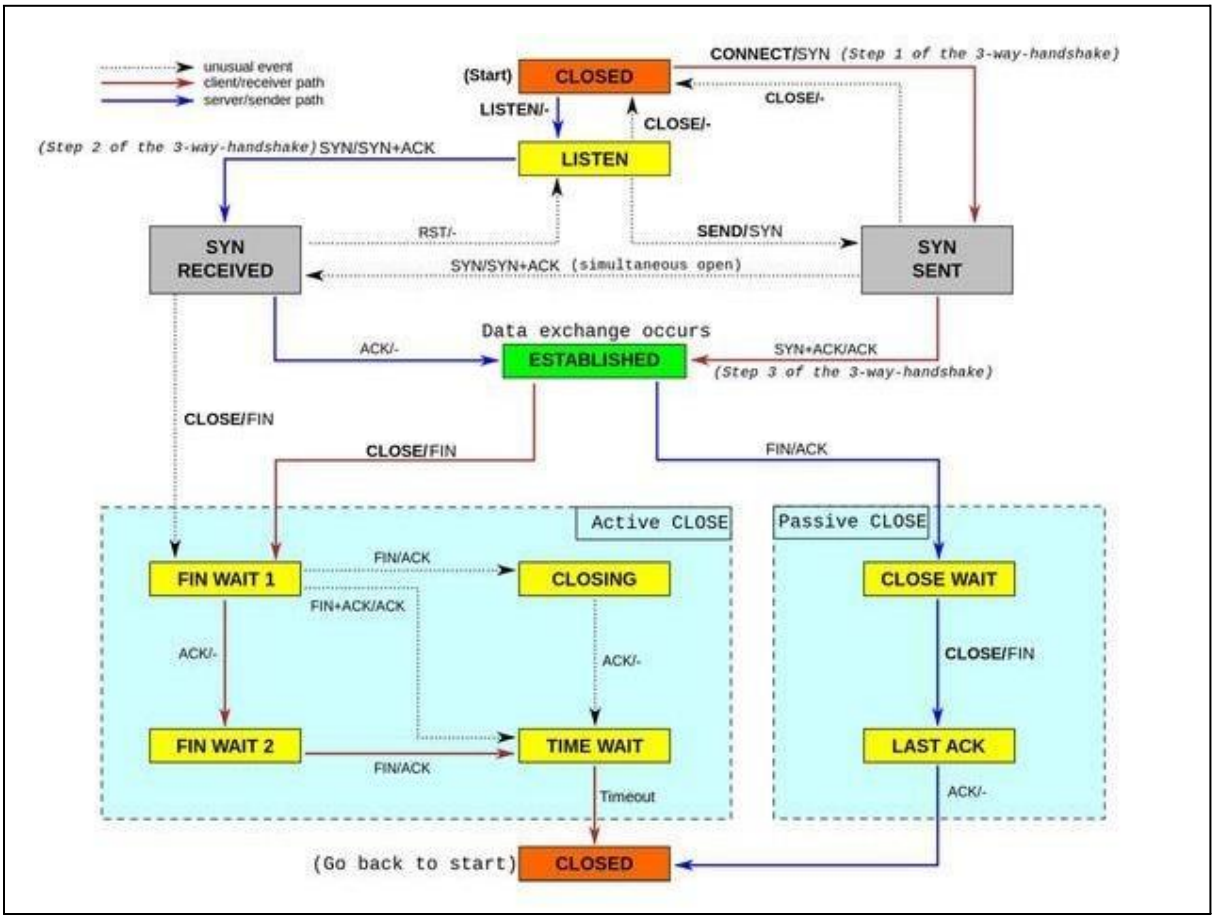
ensures that both devices are ready to communicate. The three-way handshake consists of the following steps:

- The initiating device sends a packet with the SYN (synchronize) flag set to the receiving device. This is called the "SYN packet."
- The receiving device responds with a packet that has both the SYN and ACK (acknowledge) flags set. This is called the "SYN-ACK packet."
- The initiating device then sends a packet with the ACK flag set to the receiving device. This is called the "ACK packet."

Once the three-way handshake is complete, the devices are said to be "connected" and can begin exchanging data. To ensure that data is transmitted reliably, TCP uses a flow control mechanism to prevent the sender from overwhelming the receiver. It does this by sending data in "chunks" called segments and uses a technique called "windowing" to control the amount of data that can be sent before an acknowledgement is received. When the sender sends a segment, it starts a timer and waits for an acknowledgement from the receiver. If the acknowledgement is not received before the timer expires, the sender assumes that the segment was lost and resends it.

TCP also uses a technique called "congestion control" to prevent network congestion and ensure that the network is used efficiently. When a device sends too much data too quickly, it can cause network congestion, resulting in dropped packets and slow communication. To prevent this, TCP uses a congestion control algorithm, such as the "Slow Start" algorithm, to gradually increase the amount of data that is sent.

TCP is also a reliable transport protocol, that it is ensured that all the packets sent are acknowledged. It uses a mechanism called "positive acknowledgement with retransmission" (PAR) which means that the receiver sends an acknowledgement for each packet it receives and if the sender does not receive an acknowledgement for a packet within a certain amount of time, it will retransmit the packet.



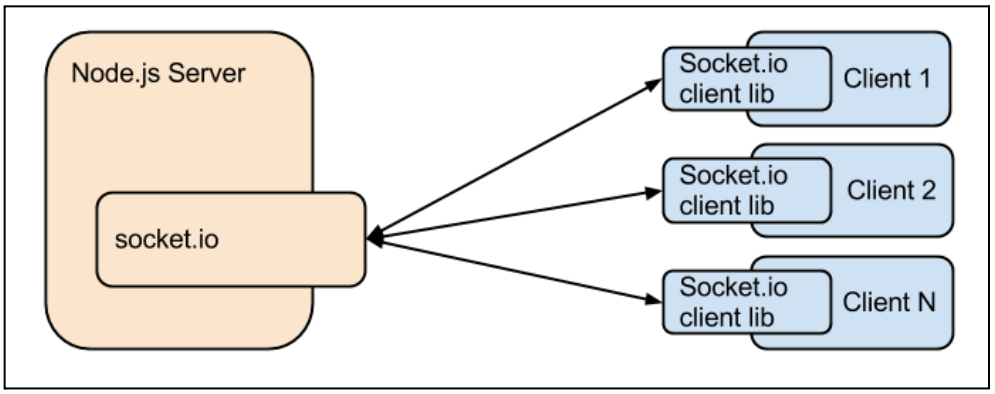
In addition, TCP is a byte stream oriented protocol. It doesn't treat the data being sent as individual packets, Instead, it views the data as a continuous stream of bytes and it is the responsibility of the application layer protocol to divide the data into chunks, if necessary.

In conclusion, TCP is a transport-layer protocol that is used to establish a reliable connection between two devices in a computer network. It is an important part of the TCP/IP protocol suite and plays a critical role in ensuring that data is transmitted reliably and efficiently over the internet. The combination of flow control, congestion control, and positive acknowledgement with retransmission, allows TCP to ensure that all packets are received and acknowledged, providing a solid foundation for internet communication

iv) Socket.io:

Socket.IO is a JavaScript library that allows real-time, bidirectional communication between web clients and servers. It is built on top of WebSockets, a technology that enables real-time, low-latency communication between web browsers and servers. However, Socket.IO adds several additional features and capabilities to WebSockets, making it a more powerful and flexible tool for building real-time web applications.

One of the key advantages of Socket.IO over regular WebSockets is its ability to automatically fall back to other, less efficient communication protocols if WebSockets are not supported by the client's browser. For example, if a client's browser does not support WebSockets, Socket.IO will automatically fall back to using long-polling, a technique where the client repeatedly sends HTTP requests to the server, asking if there is any new data to receive. This allows Socket.IO to support real-time communication in a wider variety of browsers and environments.



Another advantage of Socket.IO is its built-in support for event-based communication. With WebSockets, communication is typically done by sending messages back and forth between the client and server. With Socket.IO, communication is done by triggering and listening for events. This allows for a more organized and flexible communication structure, and makes it easier to add new features and functionality to a real-time application.

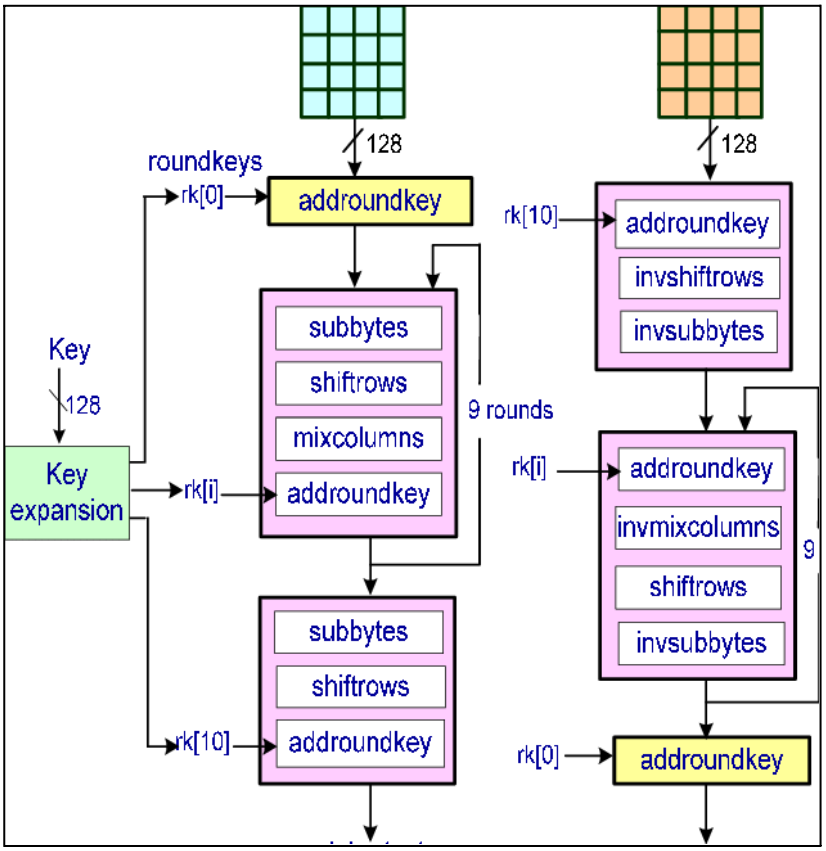
Socket.IO also provides built-in support for handling and recovering from network disconnections. It has an automatic reconnection feature for handling the clients that lose connection with the server. It also provides a mechanism to store the events and messages that were sent during a disconnected period and send them once the connection is resumed.

Additionally, Socket.IO provides a built-in system for handling namespaces and rooms. This allows you to easily divide a single Socket.IO server into multiple logical communication channels, with each channel having its own set of clients and events. This can be useful when building real-time applications with multiple rooms or chat rooms

v) AES Encryption:

AES (Advanced Encryption Standard) is a widely-used, symmetric encryption algorithm that is considered to be one of the most secure encryption methods currently in use. It was first established as a standard by the U.S. National Institute of Standards and Technology (NIST) in 2001 and it has since been adopted by organizations and governments worldwide to protect sensitive information.

AES uses a symmetric key, meaning that the same key is used to encrypt and decrypt the data. It operates on a fixed block size of 128-bits, and it supports key sizes of 128-bits, 192-bits, and 256-bits. The key size determines the number of rounds the algorithm goes through during encryption, the larger the key size the more rounds and thus more secure.



AES uses a technique called substitution-permutation network (SPN) for encryption. This technique works by dividing the data into fixed-size blocks and then applies a series of mathematical operations, such as substitution and permutation, on the data to encrypt it.

One of the key strengths of AES is that it is resistant to certain types of attacks such as known-plaintext attack and chosen-plaintext attack. AES is also a very efficient algorithm and it can be implemented in both software and hardware, making it a suitable encryption method for a wide range of applications.

```
function getEncryptedMessage() {
    const messageBox =
document.querySelector(".aes-ctr
#encrypted-message");
    let encryptedMessage = messageBox.value;
    return encryptedMessage;
}

function decryptMessage(key) {
    let encrypted = getEncryptedMessage();
    let dec = new TextDecoder();
    window.crypto.subtle.decrypt({
            name: "AES_CTR",
            counter,
            length: 64
        },
        key,
        encrypted
    ).then(function(decrypted)  {
        let decoded = dec.decode(new
Uint8Array(decrypted));
        console.log("Decrypted message: ", decoded);
    }).catch(function(err) {
        console.error(err);
    });
}
```

This code defines a getEncryptedMessage() function that retrieves the encrypted message from an HTML input element with a class of "**aes-ctr**" and an ID of **"encrypted-message".** It then defines a **decryptMessage(key)** function that takes a key as a parameter and uses the **crypto.subtle.decrypt()** method to decrypt the message.

The decrypted message is then passed to a **TextDecoder** object to be decoded from its raw binary form into a human-readable string. The decrypted message is then logged to the console for display.

It's also important to note that counter is the same counter used during the encryption process and it should be passed as the counter value to the decrypt method.

It is widely used in many standards and protocols like HTTPS, VPN, Secure Shell (SSH) and WPA/WPA2 wireless security protocol. Also, it is used in various file formats like PGP and PDF files, making it a well-established and widely used encryption standard for sensitive data

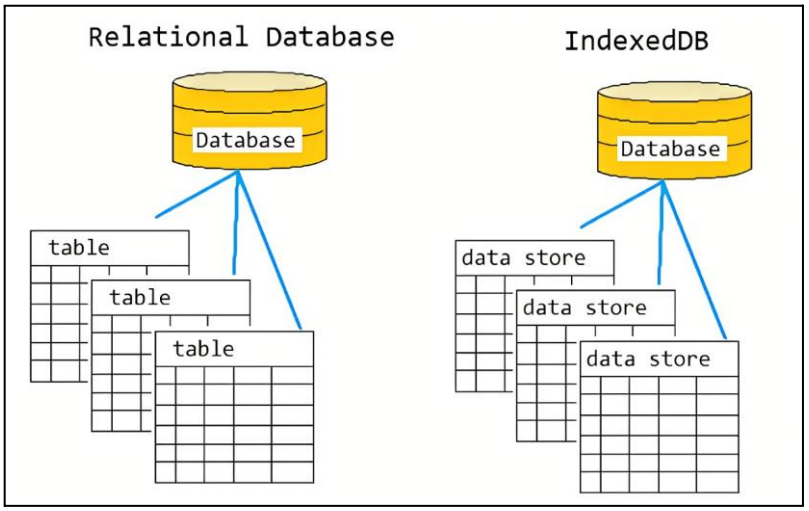
vi) IndexedDB:

IndexedDB is a JavaScript-based, object-oriented database that allows web applications to store and retrieve large amounts of structured data on the client side. It is a key-value store that allows data to be stored in an indexed manner, allowing for fast searching and retrieval. IndexedDB is a low-level API, that provides a great flexibility when storing, manipulating and querying the data, unlike more high-level APIs such as localStorage.

One of the main advantages of using IndexedDB is that it allows web applications to store large amounts of data on the client side, allowing them to work offline or in low-connectivity environments. This can greatly improve the user experience, as it allows the web application to continue functioning even if the connection to the server is lost.

Another advantage of IndexedDB is its support for indexing and searching data. It allows the developer to define indexes on specific properties of an object, which can greatly improve the performance of search queries. It also supports advanced querying mechanisms such as cursors, which allow data to be iterated and retrieved in a specific order.

IndexedDB also provides a built-in transaction mechanism that allows for atomic, consistent, isolated, and durable (ACID) transactions. This is important for maintaining the integrity and consistency of the data, and for ensuring that the database remains in a consistent state even in the face of errors or failures.



III. Proposed Architecture:

The chat application you described utilizes several key technologies to enable real-time, secure communication between users. At its core, the application is built using Node.js, a JavaScript runtime that allows developers to run JavaScript code on the server-side. This allows the application to handle multiple connections simultaneously, making it well-suited for a chat application.

The web framework Express.js is used to handle routing, middleware and other functionality, it allows to easily organize and structure different parts of the web application making it more manageable and easy to maintain.

Upon entering the chat page, the user is prompted to enter a room name. The application then checks if the room already exists, and if it does not, it creates a new room with that name. The user is then prompted to enter their name or if they have already joined the room before, the application pulls their name from a session. User profile data and room data are stored in the application's database.

Once the user joins the specified room, the chat page enables a socket connection to the server. This connection uses the WebSocket protocol, which allows for real-time, bidirectional communication between the client and the server. The server uses socket.io and APIs to manage the WebSockets and handle messages, images, files, and other data sent from the client.

The messages, images, files and data from other users are stored in the same room with the web sockets, this enables the application to have a real-time communication. The entire process runs on Node.js, and the assets such as style sheets, JavaScript, browser cache, and data to be stored in client-side storage are stored in an indexed DB, which stores all encrypted chats, files, images, etc.

When another user enters the chat page, their browser sends a network request, which is handled by a service worker, which fetches the static assets from the server. These assets are served by the server, running on Node.js and Express.js, which allows for efficient and secure communication between the client and the server.

To sum up, the chat application you have described utilizes a number of key technologies to provide real-time, secure communication between users. The application is built on Node.js, which allows it to handle multiple connections simultaneously. Express.js is used to handle routing and middleware, making the application more organized and easy to maintain. User profile data and room data are stored in the application's database and the web sockets are managed by socket.io and APIs that enable real-time communication, with the assets of the application like styles, JS, browser cache are stored in indexed DB and it allows for a

fast and responsive user experience. With this architecture the application allows the user to have secure and fast communication on a chat application

IV. Conclusion:

In the era of internet communication, privacy has become a crucial concern for users of online messaging services. With the increasing surveillance of big tech companies and government agencies, the need for a secure and private way of communicating over the internet has become more pressing than ever. In response to this, the proposed architecture in this report offers an anonymous oriented way of speaking with others over the internet.

The proposed chat application is observed to perform well, with faster speeds than many conventional chat applications currently in use. The use of AES encryption, a widely accepted and trusted method for secure communication, ensures that the chat is safe and protects against intrusions by would-be spies. This level of security is especially important for people who value their privacy or may be communicating sensitive information.

It is important to note that the proposed architecture does not only provide a secure way of communication but also an anonymous one. Anonymous communication is becoming increasingly relevant today, as people seek ways to express themselves freely without fear of repercussions. The proposed architecture provides a way for people to communicate without revealing their identities, thereby providing a safe space for people to express their thoughts, ideas, and opinions without fear of retribution.

Furthermore, the proposed architecture is versatile and can be integrated into various platforms, including mobile devices and web browsers, making it easily accessible to a wide audience. Additionally, the application can be scaled to accommodate a large number of users,

making it suitable for use in various settings, including personal, professional, and business contexts.

In conclusion, the proposed architecture offers a secure and private way of communicating over the internet, which is crucial in today's digital age. The application's use of AES encryption provides a safe environment for communication, while the anonymous orientation of the architecture allows for free expression without fear of repercussions. The proposed architecture is versatile, easily accessible, and scalable, making it a suitable solution for a wide range of users and contexts. As the need for secure and private communication continues to grow, the proposed architecture serves as a viable solution to meet that need